1. Task Description

In this assignment, you will design a single-cycle RISC-V processor in SystemVerilog. The processor must operate in-order and single-issue.

Required instruction set: RV32I custom instruction set in the HW1

31 30 29 28 27 26 25	24 23 22 21 20	19 18 17 16 15 14 13	12 11 10 9 8	7 6 5	4 3 2 1 0	
imm[31:20]		1110101	imm[19:12]		rd	LUI
imm[31:20]		1110100	imm[19:12]		rd	AUIPC
imm[20|10:1|11]		1101111	imm[19:12]		rd	JAL
imm[11:0]		1100111	rs1	000	rd	JALR
imm[12|10:5]	rs2	1100011	rs1	100	imm[4:1|11]	BEQ
imm[12|10:5]	rs2	1100011	rs1	101	imm[4:1|11]	BNE
imm[12|10:5]	rs2	1100011	rs1	000	imm[4:1|11]	BGEU
imm[12|10:5]	rs2	1100011	rs1	001	imm[4:1|11]	BLTU
imm[12|10:5]	rs2	1100011	rs1	010	imm[4:1|11]	BGE
imm[12|10:5]	rs2	1100011	rs1	011	imm[4:1|11]	BLT
imm[11:0]		1100000	rs1	000	rd	LBU
imm[11:0]		1100000	rs1	001	rd	LHU
imm[11:0]		1100000	rs1	100	rd	LB
imm[11:0]		1100000	rs1	101	rd	LH
imm[11:0]		1100000	rs1	110	rd	LW
imm[11:5]	rs2	1100001	rs1	000	imm[4:0]	SB
imm[11:5]	rs2	1100001	rs1	001	imm[4:0]	SH
imm[11:5]	rs2	1100001	rs1	010	imm[4:0]	SW
imm[11:0]		1100100	rs1	000	rd	ADDI
imm[11:0]		1100100	rs1	010	rd	SLTI
imm[11:0]		1100100	rs1	011	rd	SLTIU
imm[11:0]		1100100	rs1	100	rd	ORI
imm[11:0]		1100100	rs1	110	rd	XORI
imm[11:0]		1100100	rs1	111	rd	ANDI
0000000	imm[4:0]	1100100	rs1	001	rd	SLLI
0000000	imm[4:0]	1100100	rs1	101	rd	SRLI
0000010	imm[4:0]	1100100	rs1	101	rd	SRAI
0000000	rs2	1110001	rs1	000	rd	ADD
0000010	rs2	1110001	rs1	000	rd	SUB
0000000	rs2	1110001	rs1	001	rd	SLL
0000000	rs2	1110001	rs1	010	rd	SLT
0000000	rs2	1110001	rs1	011	rd	SLTU
0000000	rs2	1110001	rs1	100	rd	XOR
0000000	rs2	1110001	rs1	101	rd	SRL
0000010	rs2	1110001	rs1	101	rd	SRA
0000000	rs2	1110001	rs1	110	rd	OR
0000000	rs2	1110001	rs1	111	rd	AND

2. Top Module Interface

The top-level module of your design must have the following interface. Do not change the port names or directions.

```
module riscv_singlecycle
  import riscv_pkg::*;
#(
    parameter DMemInitFile = "dmem.mem",
    parameter IMemInitFile = "imem.mem"
) (
    input  logic             clk_i,       // system clock
    input  logic             rstn_i,      // system reset
    input  logic  [XLEN-1:0] addr_i,      // memory addres input for reading
    output logic             update_o,    // retire signal
    output logic  [XLEN-1:0] data_o,      // memory data output for reading
    output logic  [XLEN-1:0] pc_o,        // retired program counter
    output logic  [XLEN-1:0] instr_o,     // retired instruction
    output logic  [     4:0] reg_addr_o,  // retired register address
    output logic  [XLEN-1:0] reg_data_o,  // retired register data
    output logic  [XLEN-1:0] mem_addr_o,  // retired memory address
    output logic  [XLEN-1:0] mem_data_o,  // retired memory data
    output logic             mem_wrt_o    // retired memory write enable
);
endmodule
```

3. Memory Initialization

The instruction and data memories of the processor must be initialized from files specified via the IMemInitFile and DMemInitFile parameters respectively. Both memories must be loaded using the $readmemh .

> $readmemh(IMemInitFile, instr_mem, 0, 2047);
> $readmemh(DMemInitFile, data_mem, 0, 2047);

4. Repository Structure

The assignment repository has the following structure:

src/	Only modify files in this directory
tb/	Testbench (do not modify)
tests/	Test cases (.hex and .log files)
imem.mem	Instruction memory initialization file
dmem.mem	Data memory initialization file
Makefile	Build and test automation.
check_result.py	Automated result checker

5. Required Tools

The following tool must be installed to generate the coverage report: (Detailed expression of coverage report given in section 9.)

> sudo apt install lcov

6. Test Flow

Each test case is located in a separate subdirectory under tests/. Each subdirectory contains a .hex file (instruction memory contents) and a .log file (expected output).

Available tests:

- tests/base_test/
- tests/book_test/

Step 1 - Select a test.

Set the TK_TEST environment variable to the name of the test directory you want to use:

```
export TK_TEST=base_test
```

Step 2 - Load the instruction memory.

Copy the selected test's imem.mem file into the root directory's imem.mem. For example, for BASE_TEST, copy “tests/base_test/imem.mem” into the root imem.mem.

Note: This step can be done manually. The processor reads imem.mem via $readmemh at simulation startup, so the file must contain the correct contents before simulation begins.

Note: $ make all FLAG=<test_directory_name> (example: $ make all FLAG=base_test) will automatically copy tests/<test_directory_name>/imem.mem into the root imem.mem and $ make run will check results.

Step 3 - Compare results.

```
make run
make check
```

This command compares model.log against the golden log of the selected test. Results are reported per instruction category (R-type, I-ALU, Load, Store, Branch, Jump, etc.) as pass or fail.

7. Verifying Your Output

The make check command and the check_result.py script are provided solely as a convenience tool to help you identify differences between your output and the expected output. They are not part of the grading criteria themselves.

Your primary responsibility is to ensure that your model.log matches the golden log file (tests/<TK_TEST>/test.log) for the selected test. You are encouraged to compare these two files directly using a text comparison tool. Both VSCode and Notepad++ offer built-in file compare features that make it straightforward to spot differences line by line.

8. Writing Your Own Tests (Optional)

If you would like to write your own basic RISC-V assembly programs to test your processor, you do not need to install a toolchain. An online RISC-V assembler is available at:

```
https://riscvasm.lucasteske.dev/
```

You can write RV32I assembly directly in the browser and obtain the corresponding machine code, which can then be used as the contents of imem.mem with help of mth_hex2hex_file.sh

file in the tests folder. You can write custom C code for testing too. However, you do not have any golden.log file for your custom tests in the tests folder.

9. Coverage Report

The coverage report measures how much of your design is exercised by the tests. It tracks which lines of code and which branches (if/else, case arms) were actually executed during simulation. Lines that were never reached are highlighted, giving you a clear picture of which parts of your design have not been tested yet.

To generate the report:

```
make cov
```

The report is generated under logs/html/. Open logs/html/index.html in your browser to view the results. You can drill down into individual source files to see exactly which lines were covered.

> ***Important: Coverage results depend entirely on which test you run. For example, BASE_TEST does not contain any load or store instructions, so the load/store related branches in your design will appear as uncovered. This does not mean your implementation is wrong — it simply means those paths were not exercised by that particular test. To get higher coverage, run tests that include a wider variety of instructions.***

10. A Note on AI Tools

You are free to use AI assistants as a learning aid, but we strongly encourage you to avoid asking them to write your design from scratch. Processor design is a hands-on skill — the real learning happens when you struggle with the details, make mistakes, and figure out why something does not work. A design that passes the tests but that you do not fully understand will leave you unprepared for more advanced topics. Use AI to clarify concepts, check your reasoning, or get unstuck — not to skip the process entirely.

11. Important Notes

- The dmem.mem file must be left empty. Do not write any content into it.
- Do not add any -Wno flags to the Makefile. The design must compile without suppressing warnings.
- In case of a grading dispute, you will be required to download the exact file you submitted from Ninova, record a screen capture demonstrating the system running correctly, upload the video to YouTube as an unlisted video, and share the link.
- Don't send your code before testing.

12. Grading Criteria

Critetia	Grades
linter	10p
build	5p
run	5p
base_test	10p
Code Quality	15p
Dmem/Imem design	5p
Custom Tests	50p